

project 1

Hadeer Ghoneim

Verification Diploma

Bugs:

- The overflow and wr_ack signals were not reset properly.
→ Fixed by adding synchronous reset assignments to initialize them to 0 during reset.
- The underflow signal was implemented as a combinational output, which caused glitches.
→ Fixed by making underflow a sequential output (registered), so it holds its value for one clock cycle when asserted.
- The almostfull signal was incorrectly asserted when the FIFO count was two slots away from full (DEPTH-2).
→ Fixed by changing the condition to assert almostfull when the count equals DEPTH-1, meaning there's only one slot left.
- The count logic did not handle the case where both wr_en and rd_en were asserted simultaneously.
→ Fixed by adding conditions to correctly increment or decrement the counter when both read and write occur at the same time, depending on whether the FIFO is empty or full.

Code snippets

Assertions													
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/top/assert_reset_p	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert(@(posedge v_if_dk) (~v_if_rst_n) -> (~v_if_overflow && ~v_if_unde...)	✓
/top/v_if_dk\ /top/v_ifrst_n /top/v_ifoverflow /top/v_ifunderflow													
/top/TB/#blk#182146242#17/Immed_21	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (trans.randomize())	✓
/top/TB/#blk#182146242#28/Immed_32	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (trans.randomize())	✓
/top/TB/#blk#182146242#39/Immed_43	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (trans.randomize())	✓
/top/TB/#anonblk#182146242#50#4#/#blk#182146242#...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (trans.randomize())	✓

Activate Windows
Go to Settings to activate Windows.

```

run.do
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
4  add wave /top/v_if/*
5  run -all

```

FIFO.sv > FIFO

```
1  module FIFO(Sync_FIFO_interface.DUT v_if);
2      localparam max_fifo_addr = $clog2(v_if.FIFO_DEPTH);
3      logic [v_if.FIFO_WIDTH-1:0] mem [v_if.FIFO_DEPTH-1:0];
4      logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
5      logic [max_fifo_addr:0] count;
6
7      // Write logic - Fixed bugs
8      always @(posedge v_if.clk or negedge v_if.rst_n) begin
9          if (!v_if.rst_n) begin
10             wr_ptr <= 0;
11             v_if.wr_ack <= 0; // Fixed: wr_ack reset
12             v_if.overflow <= 0; // Fixed: overflow reset
13         end
14         else if (v_if.wr_en && !v_if.full) begin
15             mem[wr_ptr] <= v_if.data_in;
16             v_if.wr_ack <= 1;
17             wr_ptr <= wr_ptr + 1;
18         end
19         else begin
20             v_if.wr_ack <= 0;
21             if (v_if.full && v_if.wr_en)
22                 v_if.overflow <= 1;
23             else
24                 v_if.overflow <= 0;
25         end
26     end
27
28     // Read logic - Fixed bugs
29     always @(posedge v_if.clk or negedge v_if.rst_n) begin
30         if (!v_if.rst_n) begin
31             rd_ptr <= 0;
32             v_if.underflow <= 0; // Fixed: underflow is now sequential
33         end
34         else if (v_if.rd_en && !v_if.empty) begin
35             v_if.data_out <= mem[rd_ptr];
36             rd_ptr <= rd_ptr + 1;
37         end
38     end
39 endmodule
```

```

37     end
38     else begin // Fixed: underflow handling
39         if (v_if.empty && v_if.rd_en)
40             v_if.underflow <= 1;
41         else
42             v_if.underflow <= 0;
43     end
44 end
45
46 // Count logic - Fixed simultaneous R/W handling
47 always @(posedge v_if.clk or negedge v_if.rst_n) begin
48     if (!v_if.rst_n) begin
49         count <= 0;
50     end
51     else begin
52         if ({v_if.wr_en, v_if.rd_en} == 2'b10) && !v_if.full)
53             count <= count + 1;
54         else if ({v_if.wr_en, v_if.rd_en} == 2'b01) && !v_if.empty)
55             count <= count - 1;
56         else if ({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.full) // Fixed: simultaneous R/W
57             count <= count - 1;
58         else if ({v_if.wr_en, v_if.rd_en} == 2'b11) && v_if.empty) // Fixed: simultaneous R/W
59             count <= count + 1;
60     end
61 end
62
63 // Combinational outputs - Fixed almostfull bug
64 assign v_if.full = (count == v_if.FIFO_DEPTH);
65 assign v_if.empty = (count == 0);
66 assign v_if.almostfull = (count == v_if.FIFO_DEPTH - 1); // Fixed: DEPTH-1 not DEPTH-2
67 assign v_if.almostempty = (count == 1);
68

```

```

70 // Assertions with conditional compilation
71 property reset_internal;
72 | @(posedge v_if.clk) !v_if.rst_n | => ((count == 0) && (rd_ptr == 0) && (wr_ptr == 0));
73 endproperty
74 assert property (reset_internal);
75 cover property (reset_internal);
76
77 property write_acknowledge_p;
78 | @(posedge v_if.clk) disable iff(!v_if.rst_n)
79 | (v_if.wr_en && !v_if.full) | => v_if.wr_ack;
80 endproperty
81 assert property (write_acknowledge_p);
82 cover property (write_acknowledge_p);
83
84 property overflow_p;
85 | @(posedge v_if.clk) disable iff(!v_if.rst_n)
86 | (v_if.wr_en && v_if.full) | => v_if.overflow;
87 endproperty
88 assert property (overflow_p);
89 cover property (overflow_p);
90
91 property underflow_p;
92 | @(posedge v_if.clk) disable iff(!v_if.rst_n)
93 | (v_if.rd_en && v_if.empty) | => v_if.underflow;
94 endproperty
95 assert property (underflow_p);
96 cover property (underflow_p);
97
98 property empty_p;
99 | @(posedge v_if.clk) disable iff(!v_if.rst_n)
100 | (count == 0) | -> v_if.empty;
101 endproperty
102 assert property (empty_p);
103 cover property (empty_p);
104

```

```

104
105     property full_p;
106         @(posedge v_if.clk) disable iff(!v_if.rst_n)
107             (count == v_if.FIFO_DEPTH) |-> v_if.full;
108     endproperty
109     assert property (full_p);
110     cover property (full_p);
111
112     property almostfull_p;
113         @(posedge v_if.clk) disable iff(!v_if.rst_n)
114             (count == v_if.FIFO_DEPTH - 1) |-> v_if.almostfull;
115     endproperty
116     assert property (almostfull_p);
117     cover property (almostfull_p);
118
119     property almostempty_p;
120         @(posedge v_if.clk) disable iff(!v_if.rst_n)
121             (count == 1) |-> v_if.almostempty;
122     endproperty
123     assert property (almostempty_p);
124     cover property (almostempty_p);
125
126     property read_ptr_wrap_around_p;
127         @(posedge v_if.clk) disable iff(!v_if.rst_n)
128             (rd_ptr == v_if.FIFO_DEPTH - 1) && !v_if.empty && v_if.rd_en |=> (rd_ptr == 0);
129     endproperty
130     assert property (read_ptr_wrap_around_p);
131     cover property (read_ptr_wrap_around_p);
132
133     property write_ptr_wrap_around_p;
134         @(posedge v_if.clk) disable iff(!v_if.rst_n)
135             (wr_ptr == v_if.FIFO_DEPTH - 1) && !v_if.full && v_if.wr_en |=> (wr_ptr == 0);
136     endproperty
137     assert property (write_ptr_wrap_around_p);
138     cover property (write_ptr_wrap_around_p);
139

```

```
139
140     property wr_ptr_p;
141         @(posedge v_if.clk) disable iff(!v_if.rst_n)
142             (wr_ptr < v_if.FIFO_DEPTH);
143     endproperty
144     assert property (wr_ptr_p);
145     cover property (wr_ptr_p);
146
147     property rd_ptr_p;
148         @(posedge v_if.clk) disable iff(!v_if.rst_n)
149             (rd_ptr < v_if.FIFO_DEPTH);
150     endproperty
151     assert property (rd_ptr_p);
152     cover property (rd_ptr_p);
153
154     property count_p;
155         @(posedge v_if.clk) disable iff(!v_if.rst_n)
156             (count <= v_if.FIFO_DEPTH);
157     endproperty
158     assert property (count_p);
159     cover property (count_p);
160
161 endmodule
```

FIFO_coverage sv > {} FIFO_coverage

```
1  package FIFO_coverage;
2      import FIFO_transaction::*;
3      import shared_pkg::*;
4
5      class FIFO_coverage;
6          FIFO_transaction F_cvg_txn;
7
8          covergroup FIFO_cg;
9              cp_write_en: coverpoint F_cvg_txn.wr_en {
10                 bins wr_en_zero = {1'b0};
11                 bins wr_active = {1'b1};
12             }
13             cp_read_en: coverpoint F_cvg_txn.rd_en {
14                 bins rd_en_zero = {1'b0};
15                 bins rd_active = {1'b1};
16             }
17             cp_full: coverpoint F_cvg_txn.full {
18                 bins full_asserted = {1'b1};
19                 bins full_deasserted = {1'b0};
20             }
21             cp_empty: coverpoint F_cvg_txn.empty {
22                 bins empty_asserted = {1'b1};
23                 bins empty_deasserted = {1'b0};
24             }
25             cp_almost_full: coverpoint F_cvg_txn.almostfull {
26                 bins almostfull_asserted = {1'b1};
27                 bins almostfull_deasserted = {1'b0};
28             }
29             cp_almost_empty: coverpoint F_cvg_txn.almostempty {
30                 bins almostempty_asserted = {1'b1};
31                 bins almostempty_deasserted = {1'b0};
32             }
33             cp_overflow: coverpoint F_cvg_txn.overflow {
34                 bins overflow_asserted = {1'b1};
35                 bins overflow_deasserted = {1'b0};
36             }
37         }
```



```

37     cp_underflow: coverpoint F_cvg_txn.underflow {
38         option.weight = 0;
39         bins underflow_asserted = {1'b1};
40         bins underflow_deasserted = {1'b0};
41     }
42     cp_wr_ack: coverpoint F_cvg_txn.wr_ack {
43         option.weight = 0;
44         bins ack_asserted = {1'b1};
45         bins ack_deasserted = {1'b0};
46     }
47
48     // 7 Cross coverages as required
49     full_cv: cross cp_write_en, cp_read_en, cp_full {
50         ignore_bins redundant_full = binsof(cp_read_en.rd_active) && binsof(cp_full.full_asserted);
51     }
52     empty_cv: cross cp_write_en, cp_read_en, cp_empty;
53     almostempty_cv: cross cp_write_en, cp_read_en, cp_almost_empty;
54     almostfull_cv: cross cp_write_en, cp_read_en, cp_almost_full;
55     overflow_cv: cross cp_write_en, cp_read_en, cp_overflow {
56         ignore_bins impossible_overflow = binsof(cp_write_en.wr_en_zero) && binsof(cp_overflow.overflow_asserted);
57     }
58     underflow_cv: cross cp_write_en, cp_read_en, cp_underflow {
59         ignore_bins impossible_underflow = binsof(cp_read_en.rd_en_zero) && binsof(cp_underflow.underflow_asserted);
60     }
61     wr_ack_cv: cross cp_write_en, cp_read_en, cp_wr_ack {
62         ignore_bins impossible_wr_ack = binsof(cp_write_en.wr_en_zero) && binsof(cp_wr_ack.ack_asserted);
63     }
64 endgroup
65
66 function new();
67     FIFO_cg = new;
68 endfunction

```

Activate Win

```

69
70     function void sample_data(input FIFO_transaction F_txn);
71         F_cvg_txn = F_txn;
72         FIFO_cg.sample();
73     endfunction
74 endclass
75 endpackage

```

Covergroups									
File Bookmarks Window Help									
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/FIFO_coverage/FIFO_coverage									
TYPE FIFO_cg		100.00%	100	100.00%			auto(1)		
CVP FIFO_cg:op_write_en		100.00%	100	100.00%					
bin wr_en_zero		34130	1	100.00%					
bin wr_active		75870	1	100.00%					
CVP FIFO_cg:op_read_en		100.00%	100	100.00%					
bin rd_en_zero		65872	1	100.00%					
bin rd_active		44128	1	100.00%					
CVP FIFO_cg:op_full		100.00%	100	100.00%					
bin full_asserted		42455	1	100.00%					
bin full_deasserted		67545	1	100.00%					
CVP FIFO_cg:op_empty		100.00%	100	100.00%					
bin empty_asserted		12643	1	100.00%					
bin empty_deasserted		97337	1	100.00%					
CVP FIFO_cg:op_almost_full		100.00%	100	100.00%					
bin almostfull_asserted		31725	1	100.00%					
bin almostfull_deasserted		78275	1	100.00%					
CVP FIFO_cg:op_almost_empty		100.00%	100	100.00%					
bin almostempty_asserted		3570	1	100.00%					
bin almostempty_deasserted		106430	1	100.00%					
CVP FIFO_cg:op_overflow		100.00%	100	100.00%					
bin overflow_asserted		48017	1	100.00%					
bin overflow_deasserted		61983	1	100.00%					
CVP FIFO_cg:op_underflow		100.00%	100	100.00%					
bin underflow_asserted		10982	1	100.00%					
bin underflow_deasserted		99018	1	100.00%					
CVP FIFO_cg:op_wr_ack		100.00%	100	100.00%					
bin ack_asserted		42609	1	100.00%					
bin ack_deasserted		67391	1	100.00%					
CROSS FIFO_cg:full_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_full_deasse...		26913	1	100.00%					
bin cwr_en_zero_rd_active_full_deas...		17215	1	100.00%					
bin cwr_active_rd_en_zero_full_deas...		13191	1	100.00%					
bin cwr_active_rd_en_zero_full_asse...		35766	1	100.00%					
bin cwr_en_zero_rd_en_zero_full_de...		10226	1	100.00%					
bin cwr_en_zero_rd_en_zero_full_ass...		6689	1	100.00%					
ignore_bin_redundant_full		0	-	-					
CROSS FIFO_cg:empty_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_empty_dea...		26600	1	100.00%					
bin cwr_en_zero_rd_active_empty_d...		6546	1	100.00%					
bin cwr_active_rd_en_zero_empty_d...		48172	1	100.00%					
bin cwr_en_zero_rd_en_zero_empty_...		16019	1	100.00%					
bin cwr_active_rd_active_empty_ase...		313	1	100.00%					
bin cwr_en_zero_rd_active_empty_ase...		10669	1	100.00%					
bin cwr_active_rd_en_zero_empty_ase...		785	1	100.00%					
bin cwr_en_zero_rd_en_zero_empty_...		896	1	100.00%					
CROSS FIFO_cg:almostempty_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_almostempt...		25616	1	100.00%					
bin cwr_en_zero_rd_active_almostem...		16926	1	100.00%					
bin cwr_active_rd_en_zero_almostem...		47679	1	100.00%					
bin cwr_en_zero_rd_en_zero_almost...		16209	1	100.00%					
bin cwr_active_rd_active_almostempt...		1297	1	100.00%					
bin cwr_en_zero_rd_active_almostem...		289	1	100.00%					
bin cwr_active_rd_en_zero_almostem...		1278	1	100.00%					
bin cwr_en_zero_rd_en_zero_almost...		706	1	100.00%					
CROSS FIFO_cg:almostfull_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_almostfull_d...		5667	1	100.00%					
bin cwr_en_zero_rd_active_almostfull...		14258	1	100.00%					
bin cwr_active_rd_en_zero_almostfull...		46048	1	100.00%					
bin cwr_en_zero_rd_en_zero_almostf...		12302	1	100.00%					
bin cwr_active_rd_active_almostfull_a...		21246	1	100.00%					
bin cwr_en_zero_rd_active_almostfull...		2957	1	100.00%					
bin cwr_active_rd_en_zero_almostfull...		2909	1	100.00%					
bin cwr_en_zero_rd_en_zero_almostf...		4613	1	100.00%					
CROSS FIFO_cg:overflow_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_overflow_d...		8372	1	100.00%					
bin cwr_en_zero_rd_active_overflow...		17215	1	100.00%					
bin cwr_active_rd_en_zero_overflow...		19481	1	100.00%					
bin cwr_en_zero_rd_en_zero_overflow...		16915	1	100.00%					
bin cwr_active_rd_active_overflow_a...		18541	1	100.00%					
bin cwr_active_rd_en_zero_overflow...		29476	1	100.00%					
ignore_bin_impossible_overflow		0	-	-					
CROSS FIFO_cg:underflow_cv		100.00%	100	100.00%					
bin cwr_active_rd_active_underflow_...		26200	1	100.00%					
bin cwr_active_rd_active_underflow_...		713	1	100.00%					
bin cwr_en_zero_rd_active_underflo...		6946	1	100.00%					
bin cwr_en_zero_rd_active_underflo...		10289	1	100.00%					
bin cwr_active_rd_en_zero_underflo...		48957	1	100.00%					
bin cwr_en_zero_rd_en_zero_underf...		16915	1	100.00%					

Covergroups Filter: NoFilter

Covergroups									
File Bookmarks Window Help									
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/CROSS_FIFO_cg:empty_cv									
bin cwr_active_rd_active_empty_dea...		26600	1	100.00%					
bin cwr_en_zero_rd_active_empty_d...		6546	1	100.00%					
bin cwr_active_rd_en_zero_empty_d...		48172	1	100.00%					
bin cwr_en_zero_rd_en_zero_empty_...		16019	1	100.00%					
bin cwr_active_rd_active_empty_ase...		313	1	100.00%					
bin cwr_en_zero_rd_active_empty_ase...		10669	1	100.00%					
bin cwr_active_rd_en_zero_empty_ase...		785	1	100.00%					
bin cwr_en_zero_rd_en_zero_empty_...		896	1	100.00%					
/CROSS_FIFO_cg:almostempty_cv									
bin cwr_active_rd_active_almostempt...		25616	1	100.00%					
bin cwr_en_zero_rd_active_almostem...		16926	1	100.00%					
bin cwr_active_rd_en_zero_almostem...		47679	1	100.00%					
bin cwr_en_zero_rd_en_zero_almost...		16209	1	100.00%					
bin cwr_active_rd_active_almostempt...		1297	1	100.00%					
bin cwr_en_zero_rd_active_almostem...		289	1	100.00%					
bin cwr_active_rd_en_zero_almostem...		1278	1	100.00%					
bin cwr_en_zero_rd_en_zero_almost...		706	1	100.00%					
/CROSS_FIFO_cg:almostfull_cv									
bin cwr_active_rd_active_almostfull_d...		5667	1	100.00%					
bin cwr_en_zero_rd_active_almostfull...		14258	1	100.00%					
bin cwr_active_rd_en_zero_almostfull...		46048	1	100.00%					
bin cwr_en_zero_rd_en_zero_almostf...		12302	1	100.00%					
bin cwr_active_rd_active_almostfull_a...		21246	1	100.00%					
bin cwr_en_zero_rd_active_almostfull...		2957	1	100.00%					
bin cwr_active_rd_en_zero_almostfull...		2909	1	100.00%					
bin cwr_en_zero_rd_en_zero_almostf...		4613	1	100.00%					
/CROSS_FIFO_cg:overflow_cv									
bin cwr_active_rd_active_overflow_d...		8372	1	100.00%					
bin cwr_en_zero_rd_active_overflow...		17215	1	100.00%					
bin cwr_active_rd_en_zero_overflow...		19481	1	100.00%					
bin cwr_en_zero_rd_en_zero_overflow...		16915	1	100.00%					
bin cwr_active_rd_active_overflow_a...		18541	1	100.00%					
bin cwr_active_rd_en_zero_overflow...		29476	1	100.00%					
ignore_bin_impossible_overflow		0	-	-					
/CROSS_FIFO_cg:underflow_cv									
bin cwr_active_rd_active_underflow_...		26200	1	100.00%					
bin cwr_active_rd_active_underflow_...		713	1	100.00%					
bin cwr_en_zero_rd_active_underflo...		6946	1	100.00%					
bin cwr_en_zero_rd_active_underflo...		10289	1	100.00%					
bin cwr_active_rd_en_zero_underflo...		48957	1	100.00%					
bin cwr_en_zero_rd_en_zero_underf...		16915	1	100.00%					

Covergroups Filter: NoFilter

Covergroups										
File Bookmarks Window Help										
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment	
CROSS FIFO_cg:almostempty_cv		100.00%	100	100.00%		✓				
bin_cwr_active_rd_active,almostempt...		25616	1	100.00%		✓				
bin_cwr_en_zero_rd_active,almostem...		16926	1	100.00%		✓				
bin_cwr_active_rd_en_zero,almostem...		47679	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,almoste...		16209	1	100.00%		✓				
bin_cwr_active_rd_active,almostempt...		1297	1	100.00%		✓				
bin_cwr_en_zero_rd_active,almostem...		289	1	100.00%		✓				
bin_cwr_active_rd_en_zero,almostem...		1278	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,almoste...		706	1	100.00%		✓				
CROSS FIFO_cg:almostfull_cv		100.00%	100	100.00%		✓				
bin_cwr_active_rd_active,almostfull_d...		5667	1	100.00%		✓				
bin_cwr_en_zero_rd_active,almostfull...		14258	1	100.00%		✓				
bin_cwr_active_rd_en_zero,almostfull...		46048	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,almostf...		12302	1	100.00%		✓				
bin_cwr_active_rd_active,almostfull_a...		21246	1	100.00%		✓				
bin_cwr_en_zero_rd_active,almostfull...		2957	1	100.00%		✓				
bin_cwr_active_rd_en_zero,almostfull...		2909	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,almostf...		4613	1	100.00%		✓				
CROSS FIFO_cg:overflow_cv		100.00%	100	100.00%		✓				
bin_cwr_active_rd_active,overflow_d...		8372	1	100.00%		✓				
bin_cwr_en_zero_rd_active,overflow...		17215	1	100.00%		✓				
bin_cwr_active_rd_en_zero,overflow...		19481	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,overflow...		16915	1	100.00%		✓				
bin_cwr_active_rd_active,overflow_a...		18541	1	100.00%		✓				
bin_cwr_active_rd_en_zero,overflow...		29476	1	100.00%		✓				
ignore_bin_impossible_overflow		0	-	-		✓				
CROSS FIFO_cg:underflow_cv		100.00%	100	100.00%		✓				
bin_cwr_active_rd_active,underflow...		26200	1	100.00%		✓				
bin_cwr_en_zero_rd_active,underflo...		7113	1	100.00%		✓				
bin_cwr_en_zero_rd_active,underflo...		6946	1	100.00%		✓				
bin_cwr_en_zero_rd_active,underflo...		10269	1	100.00%		✓				
bin_cwr_active_rd_en_zero,underflo...		48957	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,underflo...		16915	1	100.00%		✓				
ignore_bin_impossible_underflow		0	-	-		✓				
CROSS FIFO_cg:cwr_ack_cv		100.00%	100	100.00%		✓				
bin_cwr_active_rd_active,ack_deasse...		7115	1	100.00%		✓				
bin_cwr_en_zero_rd_active,ack_deas...		17215	1	100.00%		✓				
bin_cwr_active_rd_en_zero,ack_deas...		26146	1	100.00%		✓				
bin_cwr_en_zero_rd_en_zero,ack_de...		16915	1	100.00%		✓				
bin_cwr_active_rd_active,ack_assert...		19798	1	100.00%		✓				
bin_cwr_active_rd_en_zero,ack_asse...		22811	1	100.00%		✓				
ignore_bin_impossible_cwr_ack		0	-	-		✓				

Activate Windows
Go to Settings to activate Windows.

Covergroups Filter: NoFilter

```

FIFO_interface.sv > Sync_FIFO_interface
1  interface Sync_FIFO_interface(input bit clk);
2      parameter FIFO_WIDTH = 16;
3      parameter FIFO_DEPTH = 8;
4
5      bit [FIFO_WIDTH-1:0] data_in, data_out;
6      bit rst_n, wr_en, rd_en, wr_ack, overflow, underflow,
7      full, empty, almostfull, almostempty;
8      event etrigger, ttrigger;
9
10     modport DUT (
11         input data_in, clk, rst_n, wr_en, rd_en,
12         output data_out, wr_ack, overflow, underflow, full, empty,
13         almostfull, almostempty
14     );
15
16     modport TB (
17         input clk, etrigger, ttrigger, data_out, wr_ack, overflow,
18         underflow, full, empty, almostfull, almostempty,
19         output data_in, rst_n, wr_en, rd_en
20     );
21
22     modport MON (
23         input clk, data_out, wr_ack, overflow, underflow, full,
24         empty, almostfull, almostempty, data_in, rst_n,
25         wr_en, rd_en, etrigger, ttrigger
26     );
27 endinterface

```

```

src_files.list
1  shared_pkg.sv
2  FIFO_transaction.sv
3  FIFO_scoreboard.sv
4  FIFO_coverage.sv
5  FIFO_interface.sv
6  FIFO.sv
7  FIFO_monitor.sv
8  FIFO_test.sv
9  top.sv
10

```

```

FIFO_monitor sv > FIFO_monitor
1  import FIFO_coverage::*;
2  import FIFO_transaction::*;
3  import FIFO_scoreboard::*;
4  import shared_pkg::*;
5
6  module FIFO_monitor(Sync_FIFO_interface.MON v_if);
7      FIFO_coverage cov;
8      FIFO_scoreboard sb;
9      FIFO_transaction trans;
10
11     initial begin
12         trans = new();
13         cov = new();
14         sb = new();
15         forever begin
16             @(negedge v_if.clk);
17             wait(v_if.etrigger.triggered);
18
19             trans.data_in = v_if.data_in;
20             trans.wr_ack = v_if.wr_ack;
21             trans.rd_en = v_if.rd_en;
22             trans.wr_en = v_if.wr_en;
23             trans.overflow = v_if.overflow;
24             trans.underflow = v_if.underflow;
25             trans.full = v_if.full;
26             trans.empty = v_if.empty;
27             trans.almostempty = v_if.almostempty;
28             trans.almostfull = v_if.almostfull;
29             trans.data_out = v_if.data_out;
30             trans.rst_n = v_if.rst_n;
31
32             -> v_if.ttrigger;
33

```

```

33
34     fork
35     begin
36         cov.sample_data(trans);
37     end
38     begin
39         sb.check_data(trans);
40     end
41     join
42
43     if (test_finished) begin
44         $display("correct_count = %d, errors_count = %d", correct_count, errors_count);
45         $stop;
46     end
47 end
48
49 endmodule

```

Label	Design requirement description	Stimulus generation	Functional coverage	Functionality check
RESET_1	On reset deassertion, count, wr_ptr, rd_ptr, wr_ack, overflow, underflow are zero/clear.	Directed at simulation start + random reset insertion (short/long).	coverpoint: reset asserted/deasserted; cross reset × count==0.	Both — assertion in FIFO.sv + scoreboard/golden model verifies memory cleared.
WRITE_1	When wr_en asserted and FIFO not full, data_in is written and wr_ack asserted for that cycle.	Directed single write + constrained-random writes where wr_en==1 and full==0.	cp_write_en, cp_wr_ack, cross (wr_en × wr_ack).	Both — assertion write_acknowledge_p + scoreboard checks mem content & wr_ack_ref.
READ_1	When rd_en asserted and FIFO not empty, data_out returns oldest element.	Directed single read + constrained-random reads where rd_en==1 and empty==0.	cp_read_en, cp_empty, cross (rd_en × data_out change).	Both — assertion (read behavior properties) + scoreboard data_out_ref check.
FULL_1	full becomes 1 when count == FIFO_DEPTH; almostfull when count == FIFO_DEPTH-1.	Constrained-random sequences that push until depth; directed fill/unfill.	cp_full, cp_almost_full, cross (write patterns × full/almostfull).	Both — combinational assign checks + scoreboard/golden sets full_ref/almostfull_ref.
EMPTY_1	empty becomes 1 when count == 0; almostempty when count == 1.	Directed drain to empty + random sequences that create single-element conditions.	cp_empty, cp_almost_empty, cross (read patterns × empty/almostempty).	Both — assertions empty_p/almostempty_p + scoreboard.
OVERFLOW_1	If wr_en while FIFO is full, overflow asserted.	Directed overflow attempt; constrained-random with wr_en skewed to high.	cp_overflow and cross (wr_en × full × overflow).	Both — overflow_p assertion + scoreboard overflow_ref increment detection.
UNDERFLOW_1	If rd_en while FIFO is empty, underflow asserted.	Directed underflow attempt; constrained-random with rd_en skewed to high.	cp_underflow and cross (rd_en × empty × underflow).	Both — underflow_p assertion + scoreboard underflow_ref check.

RW_SIM_1	Simultaneous wr_en==1 and rd_en==1 behavior: count unaffected (or changed correctly), data_in enqueued and data_out dequeued (FIFO semantics).	Constrained-random with high probability of simultaneous R/W; directed sequences for cornercases (empty/full simultaneous).	cross cp_write_en × cp_read_en × cp_full × cp_empty; special bins for sim-RW while empty/full.	Both — scoreboard models simultaneous R/W logic + assertions around count update and expected data_out.
PTR_WRAP_1	Read and write pointers wrap-around at FIFO_DEPTH-1 correctly.	Directed sequences that advance pointers to depth-1 then one more; random long-run to exercise wrap.	cover property read_ptr_wrap_around_p & write_ptr_wrap_around_p (already present).	Both — pointer wrap assertions in DUT + scoreboard checks (mem indexing and data order).
COUNT_BOUNDS_1	count never exceeds FIFO_DEPTH and never goes negative.	Random long runs; stress sequences with back-to-back writes/reads plus resets.	coverpoint: count values including 0, 1, FIFO_DEPTH-1, FIFO_DEPTH.	Both — count_p assertion + scoreboard consistency (mem.size() == count).
WR_ACK_CORR_1	wr_ack only asserted when a write actually stored data (not when full).	Directed: attempt writes when full/ not full; random drive.	cross cp_write_en × cp_wr_ack × cp_full.	Both — assertion with write_acknowledge_p + scoreboard wr_ack_ref compare.
FLAGS_TMGM_1	Flags (full, empty, almostfull, almostempty, overflow, underflow) update in correct timing relative to clock & operations.	Directed timing tests (check flags next posedge after ops); random sequences capture flag transitions.	coverpoint transitions for each flag; cross flag × op-type.	Both — assertions (full_p, empty_p, etc.) + scoreboard flag_ref matching.
DATA_INTG_1	Data-order integrity: FIFO must preserve FIFO ordering (first-in first-out) across sequences, resets, simultaneous operations.	Scoreboard-driven constrained-random sequences + directed cornercases (writes/read across reset, simultaneous R/W).	coverpoint: sequences of N writes followed by M reads and data ordering bins.	Golden model/scoreboard only — main correctness check on ordering done in FIFO_scoreboard.ref_model. (Assertions can detect obvious mismatches but scoreboard is authoritative.)
RESET_EDGE_1	Reset asserted in mid-transaction clears state and does not leave stale signals asserted (e.g., wr_ack must clear after reset).	Directed resets asserted at arbitrary cycles (during write/read/overflow/underflow).	cp_reset timing bins; cross reset × op-state (writing, reading).	Both — reset assertions (property reset_internal) + scoreboard verifies mem cleared and flags cleared.
STRESS_1	Long random stress/regression to expose rare sequences (race-like behaviors), multiple resets, and multi-cycle bursts.	Constrained-random long-run (e.g. 100k-1M cycles) with seed sweep.	Aggregate coverage: hit all coverpoints and crosses; metrics for coverpoint saturation.	Both — assertions will catch immediate violations; scoreboard will log data mismatches across long runs.
EDGE_CASE_1	Edge cases: alternating single write/read each cycle, bursts exceeding depth, write/read burst with width variation (data values), skewed probabilities for write/read.	Directed patterns + parametrized randomization (adjust WR_EN_ON_DIST / RD_EN_ON_DIST).	cross bins for burst length × flags × WR/RD ratio.	Both — assertions + scoreboard.

FIFO_scoreboard.sv > {} FIFO_scoreboard

```
1  package FIFO_scoreboard;
2      import FIFO_transaction::*;
3      import shared_pkg::*;
4
5      class FIFO_scoreboard #(parameter FIFO_WIDTH = 16, parameter FIFO_DEPTH = 8);
6          FIFO_transaction trans_inputs;
7          logic [FIFO_WIDTH-1:0] data_out_ref;
8          bit wr_ack_ref, overflow_ref, underflow_ref, full_ref, empty_ref, almostfull_ref, almostempty_ref;
9          logic [FIFO_WIDTH-1:0] mem_q[$];
10
11      function void ref_model(input FIFO_transaction trans_inputs);
12          if (!trans_inputs.rst_n) begin
13              mem_q.delete();
14              data_out_ref = 0;
15              full_ref = 0;
16              empty_ref = 1;
17              almostfull_ref = 0;
18              almostempty_ref = 0;
19              wr_ack_ref = 0;
20              overflow_ref = 0;
21              underflow_ref = 0;
22          end
23          else begin
24              if ((trans_inputs.wr_en && trans_inputs.rd_en) == 1) begin
25                  if (mem_q.size() == 0) begin
26                      // Only write when empty
27                      mem_q.push_back(trans_inputs.data_in);
28                      wr_ack_ref = 1;
29                  end
30                  else if (mem_q.size() == FIFO_DEPTH) begin
31                      // Only read when full
32                      data_out_ref = mem_q.pop_front();
33                  end
34              end
35          end
36      end
37  endclass
```



```

34         else begin
35             // Both read and write
36             mem_q.push_back(trans_inputs.data_in);
37             data_out_ref = mem_q.pop_front();
38             wr_ack_ref = 1;
39         end
40     end
41     else begin
42         if (trans_inputs.wr_en && (mem_q.size() != FIFO_DEPTH)) begin
43             mem_q.push_back(trans_inputs.data_in);
44             wr_ack_ref = 1;
45         end
46         else if (trans_inputs.rd_en && (mem_q.size() != 0)) begin
47             data_out_ref = mem_q.pop_front();
48         end
49         else begin
50             wr_ack_ref = 0;
51         end
52     end
53
54     // Calculate flags
55     full_ref = (mem_q.size() == FIFO_DEPTH);
56     empty_ref = (mem_q.size() == 0);
57     almostfull_ref = (mem_q.size() == FIFO_DEPTH - 1);
58     almostempty_ref = (mem_q.size() == 1);
59     overflow_ref = (trans_inputs.wr_en && full_ref);
60     underflow_ref = (trans_inputs.rd_en && empty_ref);
61 end
62 endfunction
63

```

```

63
64     function void check_data(input FIFO_transaction trans_inputs);
65         ref_model(trans_inputs);
66         if (trans_inputs.data_out != data_out_ref) begin
67             $display("Test failed, error here Data_out = %h, Expected = %h, rd_en = %b, wr_en = %b",
68                 trans_inputs.data_out, data_out_ref, trans_inputs.rd_en, trans_inputs.wr_en);
69             errors_count++;
70         end
71         else
72             correct_count++;
73         endfunction
74     endclass
75 endpackage

```

 shared_pkg.vh > ...

```
1 package shared_pkg;  
2  
3     bit test_finished = 0;  
4     int correct_count = 0;  
5     int errors_count = 0;  
6  
7 endpackage
```

FIFO_test.sv > FIFO_TB

```
1  import FIFO_coverage::*;
2  import FIFO_transaction::*;
3  import FIFO_scoreboard::*;
4  import shared_pkg::*;
5
6  module FIFO_TB(Sync_FIFO_interface.TB v_if);
7      FIFO_transaction trans;
8      int stopped = 79999;
9
10     initial begin
11         trans = new();
12         v_if.rst_n = 0;
13         @(negedge v_if.clk);
14         v_if.rst_n = 1;
15         @(negedge v_if.clk);
16
17         repeat (10000) begin
18             v_if.rd_en = 0;
19             v_if.wr_en = 1;
20             v_if.rst_n = 1;
21             assert(trans.randomize());
22             v_if.data_in = trans.data_in;
23             @(negedge v_if.clk);
24             ->v_if.etrigger;
25             wait(v_if.ttrigger.triggered);
26         end
27
28         repeat (10000) begin
29             v_if.rd_en = 1;
30             v_if.wr_en = 1;
31             v_if.rst_n = 1;
32             assert(trans.randomize());
33             v_if.data_in = trans.data_in;
34             @(negedge v_if.clk);
35             ->v_if.etrigger;
36             wait(v_if.ttrigger.triggered);
37         end
38     end
39 endmodule
```

```

37     end
38
39     repeat (10000) begin
40         v_if.rd_en = 1;
41         v_if.wr_en = 0;
42         v_if.rst_n = 1;
43         assert(trans.randomize());
44         v_if.data_in = trans.data_in;
45         @(negedge v_if.clk);
46         ->v_if.etrigger;
47         wait(v_if.ttrigger.triggered);
48     end
49
50     for (int i = 0; i <= stopped; ++i) begin
51         assert(trans.randomize());
52         v_if.data_in = trans.data_in;
53         v_if.rd_en = trans.rd_en;
54         v_if.wr_en = trans.wr_en;
55         v_if.rst_n = trans.rst_n;
56         @(negedge v_if.clk);
57         ->v_if.etrigger;
58         if (i != stopped)
59             wait(v_if.ttrigger.triggered);
60     end
61
62     test_finished = 1;
63 end
64 endmodule

```

```

top.sv > top
1  module top();
2      bit clk;
3
4      initial begin
5          clk = 0;
6          forever #4 clk = ~clk;
7      end
8
9      Sync_FIFO_interface v_if(clk);
10     FIFO DUT(v_if.DUT);
11     FIFO_TB TB(v_if.TB);
12     FIFO_monitor MON(v_if.MON);
13
14     property reset_p;
15         @(posedge v_if.clk) !v_if.rst_n |-> ((v_if.overflow == 0) && (v_if.underflow == 0));
16     endproperty
17     assert property(reset_p);
18     cover property(reset_p);
19
20 endmodule

```

```

FIFO_transaction.sv > {} FIFO_transaction
1  package FIFO_transaction;
2      class FIFO_transaction #(parameter FIFO_WIDTH = 16, parameter FIFO_DEPTH = 8);
3          rand bit [FIFO_WIDTH-1:0] data_in;
4          rand bit rst_n, wr_en, rd_en;
5          int RD_EN_ON_DIST, WR_EN_ON_DIST;
6          logic [FIFO_WIDTH-1:0] data_out;
7          bit wr_ack, overflow, underflow, full, empty, almostfull, almostempty;
8
9          function new(input int RD_EN_ON_DIST = 30, input int WR_EN_ON_DIST = 70);
10             this.RD_EN_ON_DIST = RD_EN_ON_DIST;
11             this.WR_EN_ON_DIST = WR_EN_ON_DIST;
12         endfunction
13
14         constraint randoms{
15             rst_n dist{ 1 :/ 98, 0 :/ 2 };
16             wr_en dist{ 1 :/ WR_EN_ON_DIST, 0 :/ (100 - WR_EN_ON_DIST) };
17             rd_en dist{ 1 :/ RD_EN_ON_DIST, 0 :/ (100 - RD_EN_ON_DIST) };
18         }
19     endclass
20 endpackage

```



